

Phase 5: Deep Learning, Computer Vision & LLMs

Part 1: The Foundations of Artificial Neural Networks (ANN)

Module 1: The Artificial Neuron (Perceptron)

- **The Perceptron:** Understanding **Weights** (importance), **Biases** (thresholds), and the summation math.
- **Activation Functions:** Introducing non-linearity via `Sigmoid`, `Tanh`, `ReLU`, and `Leaky ReLU`.
- **The Softmax Function:** Converting raw scores into probabilities for multi-class decisions.

Module 2: Multi-Layer Perceptrons (MLP) & Architecture

- **Layer Stacking:** Designing Input, Hidden, and Output layers.
- **The Forward Pass:** How data flows through the network to generate a "guess."
- **Weight Initialization:** Strategies like `Xavier` and `He` initialization to prevent training failure.

Module 3: The Math of Learning (Optimization)

- **Loss Functions:** Measuring error using Mean Squared Error (`MSE`) for regression and **Categorical Cross-Entropy** for classification.
- **Backpropagation:** Using the **Chain Rule** to find which weights caused the error.
- **Gradient Descent:** Updating weights using Stochastic Gradient Descent (`SGD`) and **Learning Rates**.
- **Advanced Optimizers:** Comparing `RMSProp` and `Adam/AdamW` for faster, more stable training.

Module 4: Deep Learning Frameworks & Tensors

- **Tensor Operations:** Moving from NumPy to Tensors. Managing CPU vs. GPU (CUDA) memory.
- **Automatic Differentiation (Autograd):** How PyTorch (primary framework) tracks math to calculate gradients automatically.
- **Computational Graphs:** Understanding static vs. dynamic graph execution.

Part 2: Computer Vision (Pixels & Patterns)

Module 5: Convolutional Neural Networks (CNN) Basics

- **The Convolution Layer:** Using **Kernels/Filters** to detect edges and textures.
- **Padding & Stride:** Controlling the output size of the image.
- **Pooling Layers:** Reducing data size using **Max Pooling** and **Average Pooling**.

Module 6: Advanced CNN Architectures

- **Deep Backbones:** Studying the evolution from **VGG** to **Inception**.
- **Residual Networks (ResNet):** Understanding **Skip Connections** to train very deep networks without signal loss.
- **Batch Normalization & Dropout:** Techniques to stabilize training and prevent overfitting.

Module 7: Object Detection (Finding Things)

- **Localization:** Predicting Bounding Boxes ($[x, y, w, h]$).
- **YOLO (You Only Look Once):** Real-time detection logic (YOLOv8 to YOLOv11).
- **Metrics:** Understanding Intersection over Union (**IoU**) and Mean Average Precision (**mAP**).

Module 8: Image Segmentation (Medical & Precision)

- **Semantic Segmentation:** Pixel-level classification with **U-Net**.
- **Skip Connections in U-Net:** Recovering spatial detail for high-precision masks.

- **Loss for Masks:** Using **Dice Loss** and **Jaccard Index** instead of standard accuracy.

Module 9: 3D Vision & Volumetric Data

- **3D Convolutions:** Processing MRI/CT scans as 3D blocks rather than 2D slices.
- **Voxel Analysis:** Calculating the volume of anomalies (tumors/lesions) automatically.
- **Depth Estimation:** How AI understands distance in images.

Part 3: Sequence Modeling & Natural Language (NLP)

Module 10: Recurrent Neural Networks (RNN)

- **Sequence Data:** Why images are different from time-series or sentences.
- **Vanishing Gradients:** Why standard RNNs forget the beginning of a sentence.
- **LSTMs & GRUs:** Using "Gates" to store long-term memory.

Module 11: Text Representations (Embeddings)

- **Tokenization:** Converting text to IDs (Word, Character, and BPE tokenization).
- **Word Vectors:** Turning words into math using **Word2Vec** and **GloVe**.
- **Latent Space:** Understanding how "Doctor" and "Hospital" end up close to each other mathematically.

Module 12: The Attention Revolution

- **Self-Attention:** How a model "looks" at all words in a sentence to understand context.
 - **The Transformer Block:** Deep dive into the Encoder-Decoder architecture.
 - **Multi-Head Attention:** Allowing the model to focus on different parts of a sentence simultaneously.
-

Part 4: Large Language Models (LLMs) & Generative AI

Module 13: LLM Architectures + Hugging Face Ecosystem

- **Decoder-Only Models:** The architecture of GPT, Llama, DeepSeek, Mistral, etc.
- **Pre-training:** How models learn the “knowledge of the internet” via Next Token Prediction.
- **Context Windows:** Understanding the limits of how much text a model can “read” at once.
- **Hugging Face Transformers as the universal standard:** pipelines, AutoModel, tokenizers, and efficient model loading.

Module 14: Tuning & Alignment

- **Instruction Tuning (SFT):** Teaching a model to follow a user's command.
- **RLHF:** Reinforcement Learning from Human Feedback—making AI safe and helpful.
- **PEFT & LoRA:** "Low-Rank Adaptation"—how to fine-tune a giant model on a cheap, local GPU.

Module 15: Retrieval Augmented Generation (RAG)

- **Vector Databases:** Storing your local PDFs and data in **ChromaDB** or **Pinecone**.
- **Semantic Search:** Using embeddings to find the "relevant" page before the LLM answers.
- **RAG Pipelines:** Designing a "Chat with your Data" system to eliminate hallucinations.

Module 16: Vision-Language Models (VLM)

- **CLIP (Contrastive Language-Image Pre-training):** How AI connects "A photo of a dog" to an actual image.

- **Visual Question Answering (VQA):** How models like GPT-4o or Llama-Vision "see" an image and talk about it.
 - **Multimodal Transformers:** Processing pixels and tokens in the same model.
-

Part 5: Agency, Deployment & Ethics (The Finish Line)

Module 17: Prompt Engineering & Reasoning

- **Chain-of-Thought (CoT):** Forcing the AI to "think step-by-step" to solve math or logic.
- **Few-Shot Prompting:** Giving the AI examples within the prompt to improve performance.
- **System Prompts:** Setting the "Identity" and "Constraints" of the AI.

Module 18: Agentic AI & Autonomous Workflows

- **Tool Use (Function Calling):** Teaching an LLM how to use a Calculator, Search Engine, or Python script.
- **Multi-Agent Systems:** Using **LangChain** or **CrewAI** to make different AIs work together.
- **Planning & Memory:** How agents keep track of long-term goals.

Module 19: Model Optimization & Quantization

- **Model Shrinking:** Using **4-bit Quantization** (\$GGUF/EXL2\$) to run a 70B model on a local laptop.
- **ONNX & TensorRT:** Converting models for industrial-speed inference.
- **Pruning:** Removing useless neurons to make models faster and lighter.

Module 20: Ethical AI, Bias & Safety

- **Bias Detection:** Identifying when medical or social models are "unfair."
- **Hallucination Mitigation:** Strategies to ensure AI tells the truth.

- **Deployment Safety:** Building guardrails to prevent AI from generating harmful content.

Module 21: Generative & Multimodal AI (2026 Must-Have)

- **Diffusion models & Stable Diffusion:** Image generation for crop health visualization, medical report augmentation, or synthetic KPK agricultural data.
- **Modern multimodal models (LLaVA, Florence-2, CLIP successors).**
- **Evaluation in the LLM era:** RAGAS, faithfulness metrics, and LLM-as-judge (instead of relying only on accuracy/F1).

Phase 5: Deep Learning & AI Systems Tasks

Task 1: The Binary Logic Perceptron

- **Topic:** Module 1 (The Neuron).
- **Goal:** Manually build a single neuron to solve the "AND" or "OR" logic gate problem.
- **Requirements:** Initialize weights w_1 , w_2 and bias b manually. Use the Step function as the activation.
- **Deliverable:** A script that correctly outputs 1 for $(1, 1)$ and 0 for others without using any DL libraries.

Task 2: The MNIST "Digit Master" (MLP)

- **Topic:** Module 2 (Multi-Layer Perceptrons).
- **Goal:** Build a 3-layer MLP to classify handwritten digits (0–9).
- **Requirements:** Include an Input layer (784 neurons), one Hidden layer (128 neurons), and an Output layer (10 neurons with Softmax).
- **Deliverable:** A trained model achieving $>92\%$ accuracy on the MNIST test set.

Task 3: The Optimizer Battle (Math of Learning)

- **Topic:** Module 3 (Optimization).
- **Goal:** Compare the convergence speed of \$SGD\$ vs. \$Adam\$.
- **Requirements:** Train the same MLP architecture twice. Keep the learning rate identical for both.
- **Deliverable:** A Matplotlib line graph showing the "Loss Curve" comparison. Explain why one is faster.

Task 4: The Tensor Benchmark

- **Topic:** Module 4 (Frameworks & Tensors).
- **Goal:** Measure the performance gain of GPU (CUDA) over CPU.
- **Requirements:** Perform a matrix multiplication of two \$5000 \times 5000\$ tensors.
- **Deliverable:** A table showing the time taken on CPU vs. GPU. Use `torch.cuda.is_available()` to verify hardware.

Task 5: The CNN "Feature Finder"

- **Topic:** Module 5 (CNN Basics).
- **Goal:** Build a basic CNN and visualize what the "Filters" are actually seeing.
- **Requirements:** Implement 2 Conv2D layers and 2 MaxPool layers.
- **Deliverable:** Use `hook` or `activation_map` to show an image of the "edges" the first layer detected.

Task 6: The "Deep" ResNet Classifier

- **Topic:** Module 6 (Advanced CNNs).
- **Goal:** Use a deep backbone to classify 10 categories of real-world objects (CIFAR-10).
- **Requirements:** Implement **Skip Connections** (Residual Blocks) or use a pre-trained ResNet-18. Apply **Dropout** (0.5).
- **Deliverable:** A script that saves the model weights as a .pth or .h5 file.

Task 7: The Real-Time Security Bot

- **Topic:** Module 7 (Object Detection).
- **Goal:** Detect multiple people and objects in a live video stream.
- **Requirements:** Use the **YOLOv11** nano model. Run it at >20 FPS on a standard laptop.
- **Deliverable:** A video output with real-time bounding boxes and confidence scores.

Task 8: The "Surgeon's Assistant" (Segmentation)

- **Topic:** Module 8 (Image Segmentation).
- **Goal:** Isolate specific regions (e.g., lungs or a tumor) in a 2D medical slice.
- **Requirements:** Build a **U-Net** architecture. Use **Dice Loss** as the optimization objective.
- **Deliverable:** A "Predicted Mask" image overlaid on the original medical scan.

Task 9: Volumetric Brain Analysis (3D Vision)

- **Topic:** Module 9 (3D Vision).
- **Goal:** Process a 3D MRI scan (NIfTI format) to detect anomalies.
- **Requirements:** Use **3D Convolutions** (Conv3D). Slice the volume into 3D patches ($64 \times 64 \times 64$).
- **Deliverable:** A model that outputs a 3D segmentation map.

Task 10: The Long-Memory Sequence Predictor

- **Topic:** Module 10 (RNN/LSTM).
- **Goal:** Predict the next word in a sentence or the next value in a stock price.
- **Requirements:** Implement an **LSTM** layer to handle long-range dependencies.
- **Deliverable:** A text-generation script that completes a half-finished sentence.

Task 11: The Semantic Space Map (Embeddings)

- **Topic:** Module 11 (NLP Embeddings).
- **Goal:** Prove that the AI understands the relationship between words.

- **Requirements:** Extract weights from an Embedding layer. Use **Cosine Similarity** to find the 5 words closest to "Hospital."
- **Deliverable:** A 2D scatter plot (using PCA) showing clusters of related words.

Task 12: Building "Self-Attention"

- **Topic:** Module 12 (Transformers).
- **Goal:** Code the \$Query, Key, Value\$ (\$Q, K, V\$) mechanism from scratch.
- **Requirements:** Implement the formula: $\text{Attention}(Q, K, V) = \frac{\text{softmax}(QK^T/\sqrt{d_k})V}{\sum V}$.
- **Deliverable:** A function that takes a word vector and outputs the "Attention Weights" for a 5-word sentence.

Task 13: The Local "Llama" Deployment

- **Requirements:** Use Hugging Face + bitsandbytes (4-bit quantization) with Ollama or transformers pipeline. Ensure the model runs without an internet connection.
- **Deliverable:** A screen recording of the local model answering a complex logic question.

Task 14: Specialized LLM (Fine-Tuning)

- **Topic:** Module 14 (LLM Tuning).
- **Goal:** Teach an LLM a specific local dialect or a very narrow medical terminology.
- **Requirements:** Use **LoRA** (Low-Rank Adaptation) to fine-tune a model on a dataset of 500+ examples.
- **Deliverable:** A "Before vs. After" comparison of the model's answers on the specialized topic.

Task 15: The "Private Librarian" (RAG)

- **Topic:** Module 15 (Retrieval Augmented Generation).
- **Goal:** Chat with a custom folder of PDFs (e.g., medical research papers).
- **Requirements:** Use a **Vector Database** (ChromaDB). Implement a "Retrieval" step before the LLM generates the answer.

- **Deliverable:** A system that provides the answer *and* the source (page number) of the information.

Task 16: The "Seeing" AI (Multimodal)

- **Topic:** Module 16 (Vision-Language).
- **Goal:** Input an image and have the AI describe it in detail.
- **Requirements:** Use a pre-trained **CLIP** or **Llama-3-Vision** model.
- **Deliverable:** A script that takes a JPG and outputs a text description (e.g., "A chest X-ray showing mild congestion").

Task 17: The Reasoning Lab (Prompt Engineering)

- **Topic:** Module 17 (Prompting).
- **Goal:** Force an LLM to solve a math word problem it usually fails.
- **Requirements:** Compare a "Zero-shot" prompt vs. a "**Chain-of-Thought**" prompt.
- **Deliverable:** A document showing how "Step-by-step" reasoning improved the accuracy.

Task 18: The Autonomous Research Agent

- **Topic:** Module 18 (Agentic AI).
- **Goal:** Build an agent that can search the web and summarize news.
- **Requirements:** Use **LangChain** or **CrewAI**. Give the agent a "Search Tool" and a "Writer Tool."
- **Deliverable:** An agent that, when asked "What happened in KP today?", searches the web and writes a 3-paragraph summary.

Task 19: The "Edge" AI Compressor

- **Topic:** Module 19 (Model Optimization).
- **Goal:** Shrink an LLM so it fits on a low-end device (e.g., a phone or old laptop).
- **Requirements:** Apply **4-bit Quantization** (\$GGUF\$). Convert a 15GB model to \$<5\$GB.
- **Deliverable:** A report showing the "Perplexity" (accuracy) loss vs. the memory saved.

Task 20: The Truth Auditor (Ethics)

- **Topic:** Module 20 (Ethics & Safety).
- **Goal:** Test a medical AI for "Hallucinations" and "Bias."
- **Requirements:** Create a "Stress Test" dataset of 50 questions where the answer is "I don't know."
- **Deliverable:** A scorecard showing how many times the AI "lied" vs. admitted it didn't have the data.